





ABACuS

All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation

Ataberk Olgun
Yahya Can Tuğrul
F. Nisa Bostancı
İsmail Emir Yüksel
Haocong Luo
Steve Rhyner
A. Giray Yağlıkçı
Geraldo F. Oliveira
Onur Mutlu

SAFARI
ETH zürich

Problem & Goal

Problem

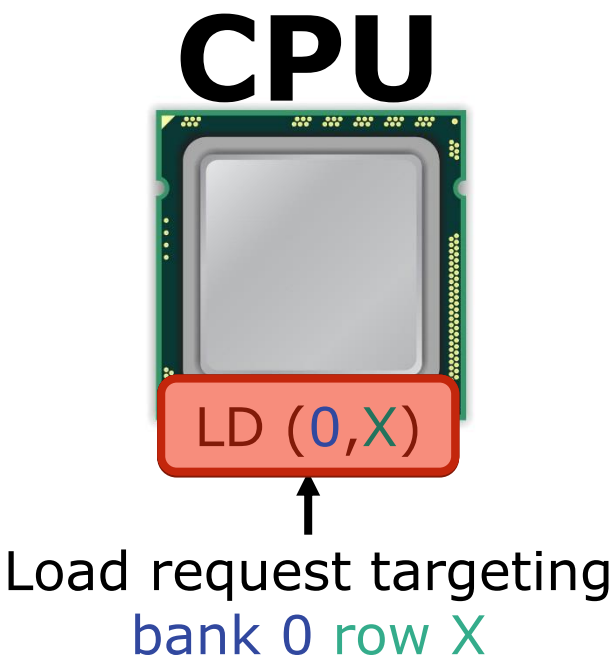
No existing mitigation technique prevents RowHammer bitflips
at low area, performance and energy costs

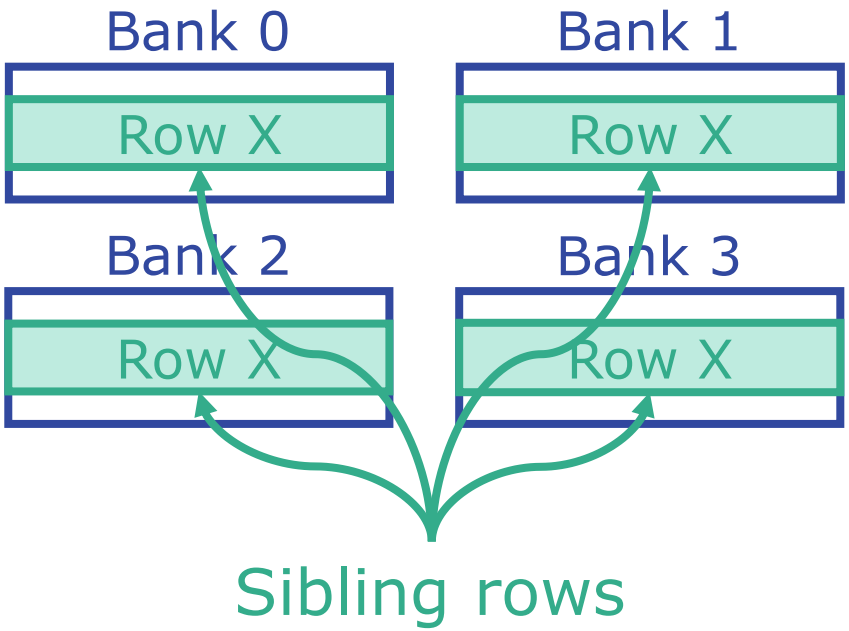
Goal

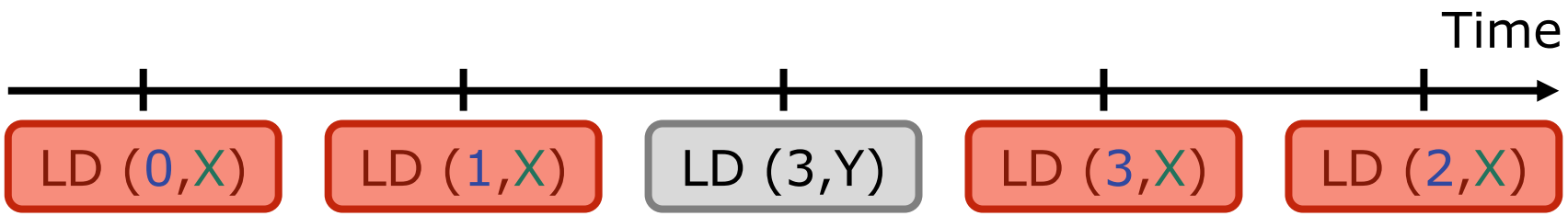
Prevent RowHammer bitflips
at low performance, energy, and area cost
especially at very low RowHammer thresholds
(e.g., 125 aggressor row activations induce a bitflip)

Key Observation

Many workloads access the same row address in different banks at around the same time







Explanation for the Key Observation

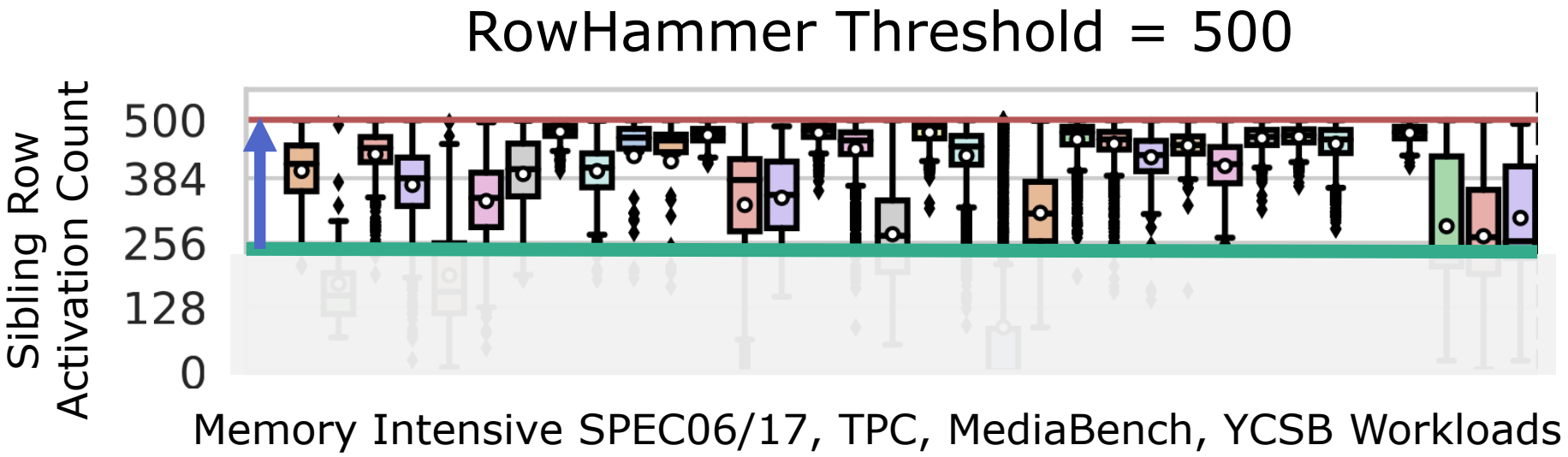
1 Spatial locality in memory accesses
(e.g., [Smith+, ACM CSUR 1982])

- A program tends to access neighboring cache blocks at around the same time
- e.g., a streaming access to an array

2 Modern physical → DRAM address mappings
(e.g., [Pessl+, USENIX Security 2016 and Kaseridis+, MICRO 2011])

- Place neighboring cache blocks into different banks, but into the same row
- Leverage DRAM bank-level parallelism for higher-throughput DRAM access

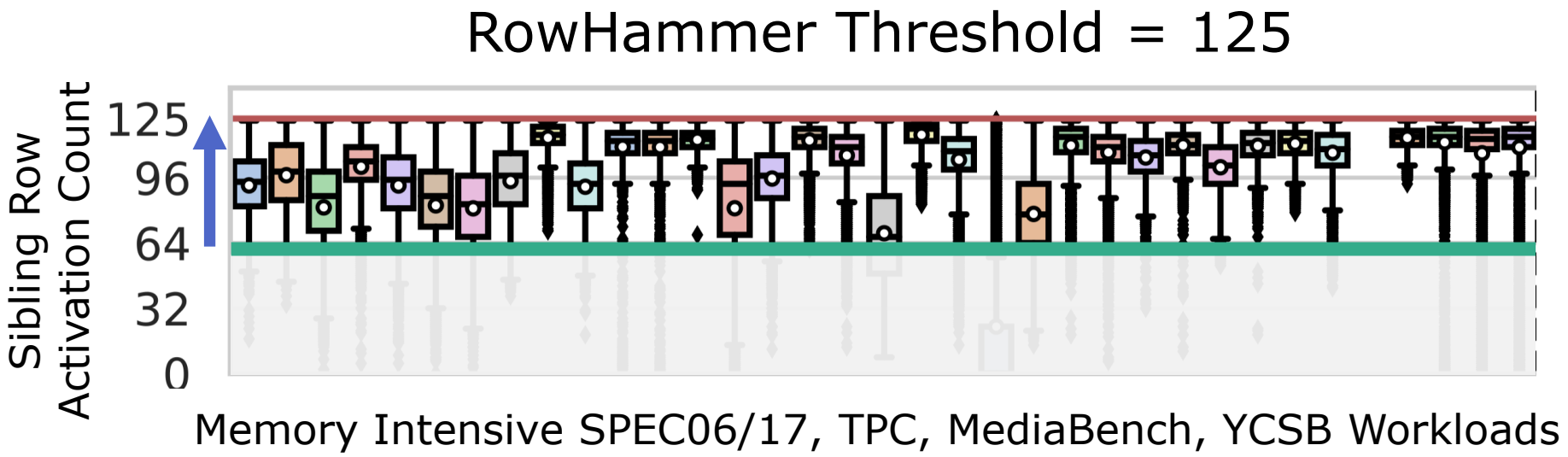
Sibling Row Activation Count for RowHammer Threshold = 500



If a row is activated 500 times
its siblings are likely activated more than 250 times

The sibling row with the highest activation count
yields a good estimate for the activation count of all siblings

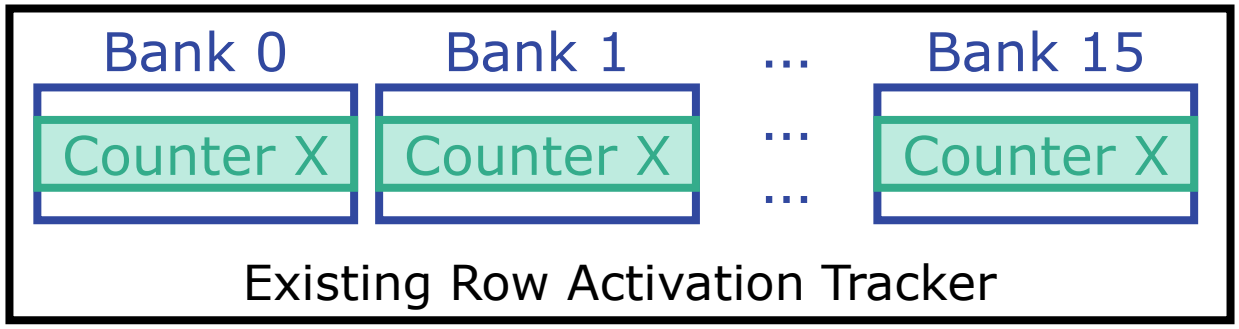
Sibling Row Activation Count for RowHammer Threshold = 125



The sibling row with the highest activation count yields an even better estimate for the activation count of all sibling rows

Existing Per-Bank Activation Counters Induce High Storage Overhead

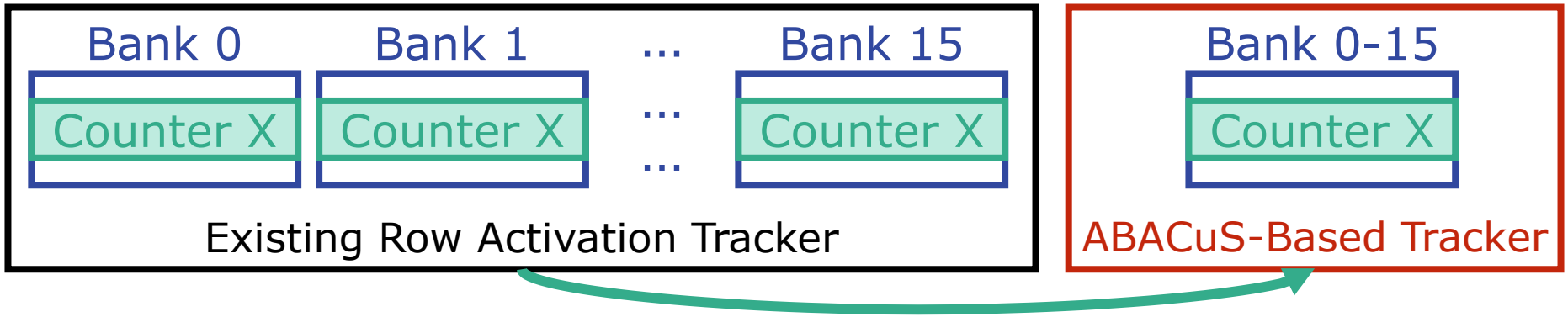
- There are many (e.g., 16) banks in a DRAM chip
  - Newer DRAM standards (DDR5) have more (32) banks
  - # of activation counters linearly increases with # of banks



Need twice as many counters for DDR5

ABACuS: Key Idea

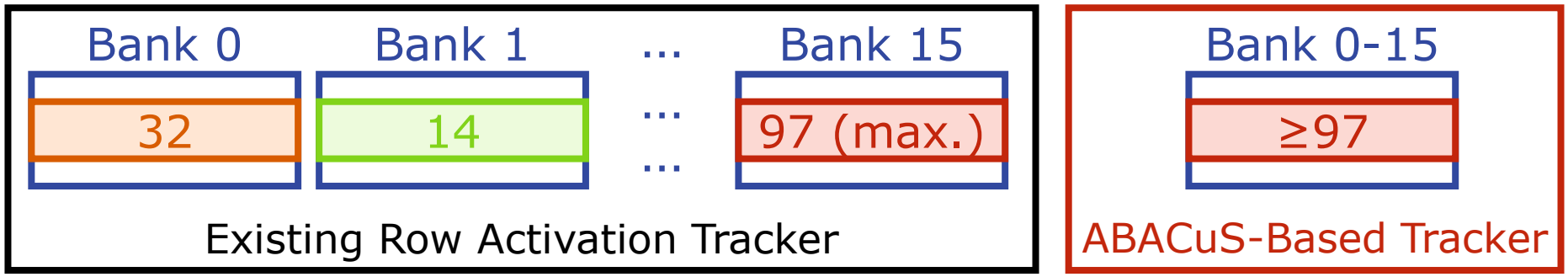
- There are many (e.g., 16) banks in a DRAM chip
  - Newer DRAM standards (DDR5) have more (32) banks
  - # of activation counters linearly increases with # of banks
- Sibling rows have similar activation counts
- Have one counter for all siblings
  - Reduce the number of counters by a factor of the number of banks



16x reduction in number of counters

Maximum Activation Count

- Track the maximum (worst) activation count across all sibling rows using one counter



ABACuS counter value vs. maximum activation count

If ABACuS counter is smaller

- Cannot preventively refresh on time
- Cannot mitigate bitflips
- Not secure

If ABACuS counter is larger

- Unnecessary preventive refreshes
- Higher perf. and energy overheads
- Lower performance

Our design goal

ABACuS counter value == maximum activation count

ABACuS: Implementation



ABACuS: All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation

Ataberk Olgun Yahya Can Tugrul Nisa Bostanci Ismail Emir Yuksel
Haocong Luo Steve Rhyner Abdullah Giray Yaglikci Geraldo F. Oliveira Onur Mutlu

ETH Zurich

We introduce ABACuS, a new low-cost hardware-counter-based RowHammer mitigation technique that performance-, energy-, and area-efficiently scales with worsening RowHammer vulnerability. We observe that both benign workloads and RowHammer attacks tend to access DRAM rows with the same row address in multiple DRAM banks at around the same time. Based on this observation, ABACuS's key idea is to use a single shared row activation counter *to track activations to the rows with the same row address in all DRAM banks. Unlike state-of-the-art RowHammer mitigation mechanisms that implement a separate row activation counter for each DRAM bank, ABACuS implements fewer counters (e.g.,* only *one) to track an equal number of aggressor rows.*

RowHammer threshold (N_{RH}), has reduced by more than an order of magnitude in less than a decade [14].[1] As many prior works demonstrate on real systems [1,2,4,15,20–83], RowHammer bitflips can lead to security exploits that 1) take over a system, 2) leak security-critical or private data, and 3) manipulate safety-critical applications' behavior in undesirable ways. As a result, a large body of work [1,15,19,38,44,55,84–88,88–135] proposes mitigation mechanisms to prevent RowHammer bitflips.

Key Problem. Many prior works (e.g., [1,98,102,106,107,110,112,116,117,125,134,135]) propose using a set of counters to track the activation counts of potential aggressor rows (counter-based mechanisms). Using counters to determine

https://arxiv.org/pdf/2310.09977.pdf

Evaluation Methodology

- **Performance and energy consumption evaluation:** Cycle-level simulations using Ramulator [Kim+, CAL 2015] and DRAMPower [Chandrasekar+, DATE 2013]

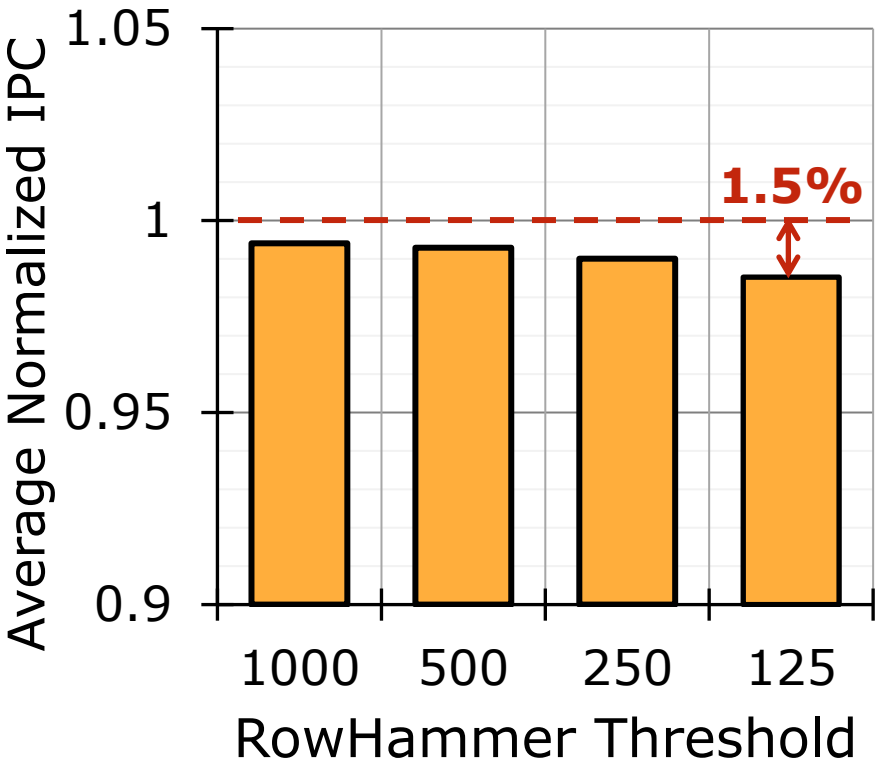
- **System Configuration:**

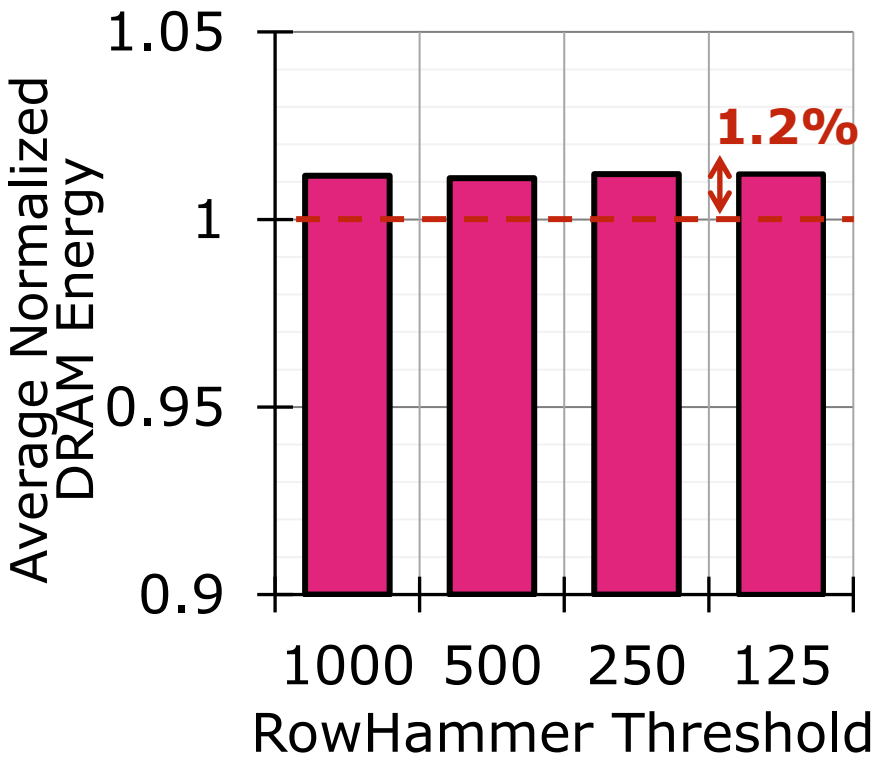
Processor	1 or 8 cores, 3.6GHz clock frequency, 4-wide issue, 128-entry instruction window
DRAM	DDR4, 1 channel, 2 rank/channel, 4 bank groups, 4 banks/bank group, 128K rows/bank, 3200 MT/s
Memory Ctrl.	64-entry read and write requests queues, Scheduling policy: FR-FCFS with a column cap of 16 Last-Level Cache 2 MiB (single-core), 16 MiB (8-core)

- **Comparison Points:** 4 state-of-the-art RowHammer mitigations
  - Graphene (best performing), Hydra (area-optimized best performing), Low Processor Chip Area Cost: REGA, PARA

- **Workloads:** 62 1- & 8-core (multiprogrammed) workloads
  - SPEC CPU2006, SPEC CPU2017, TPC, MediaBench, YCSB

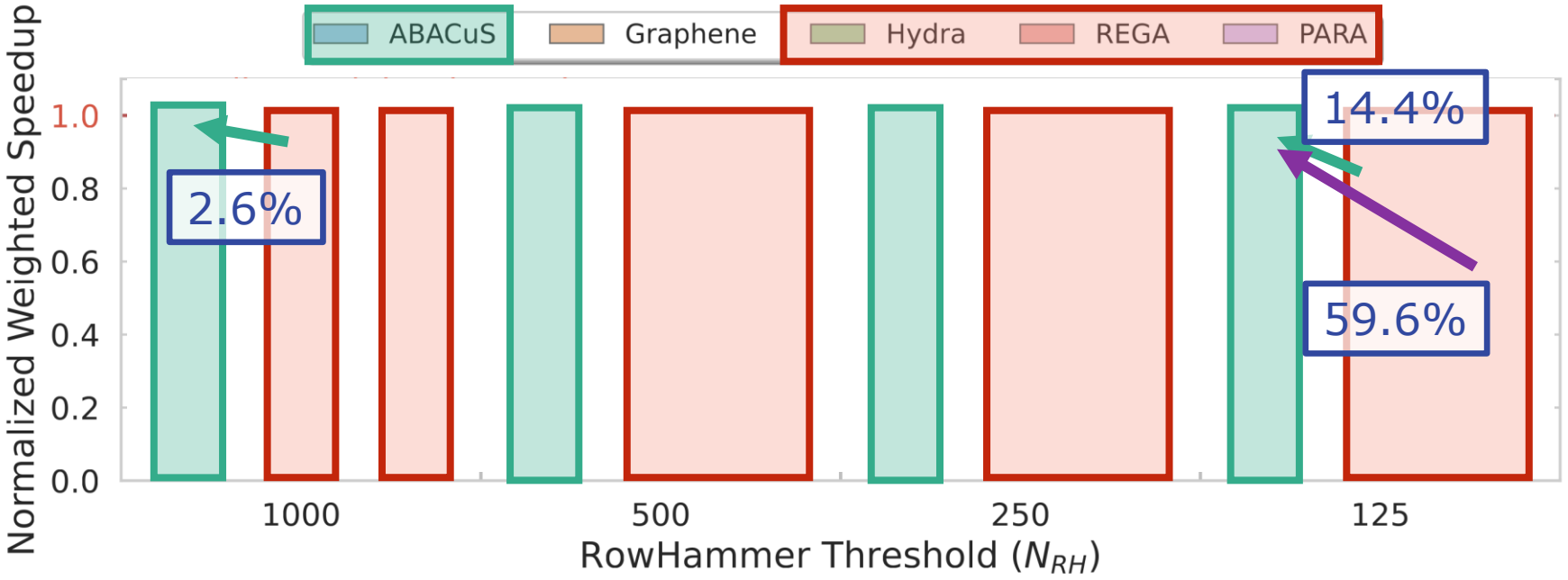
Single-Core Performance and Energy





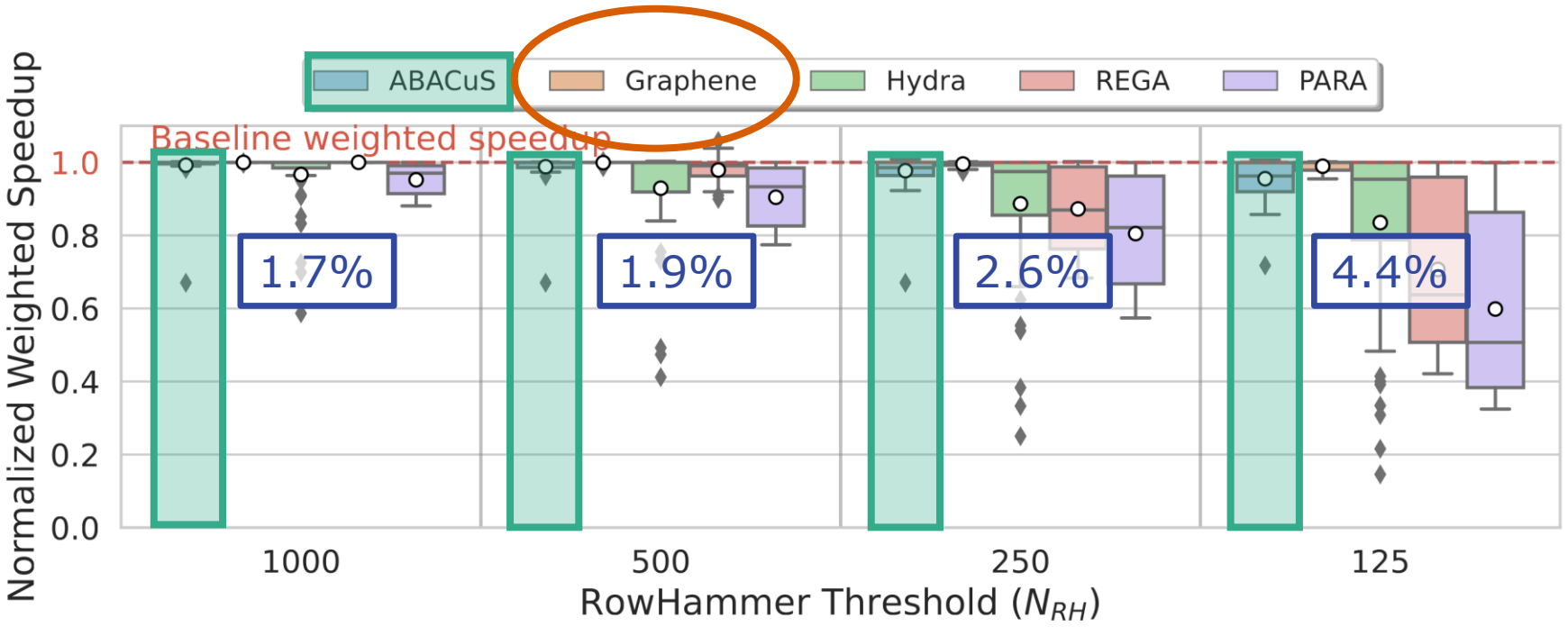
ABACuS prevents bitflips with very small
average performance and DRAM energy overheads
compared to a baseline system with *no* RowHammer mitigation

8-Core Performance Comparison



ABACuS outperforms Hydra and PARA at all RowHammer thresholds

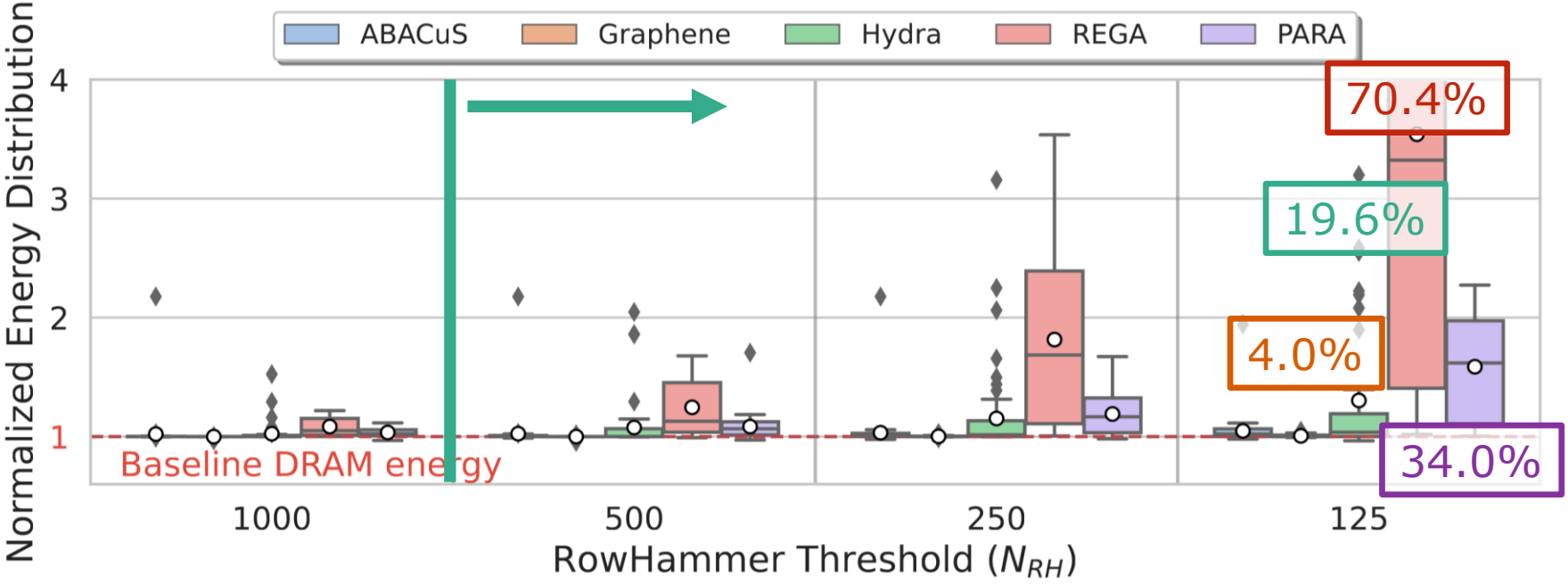
8-Core Performance Comparison



ABACuS outperforms Hydra and PARA at all RowHammer thresholds

ABACuS incurs a small performance overhead over Graphene

8-Core DRAM Energy Comparison

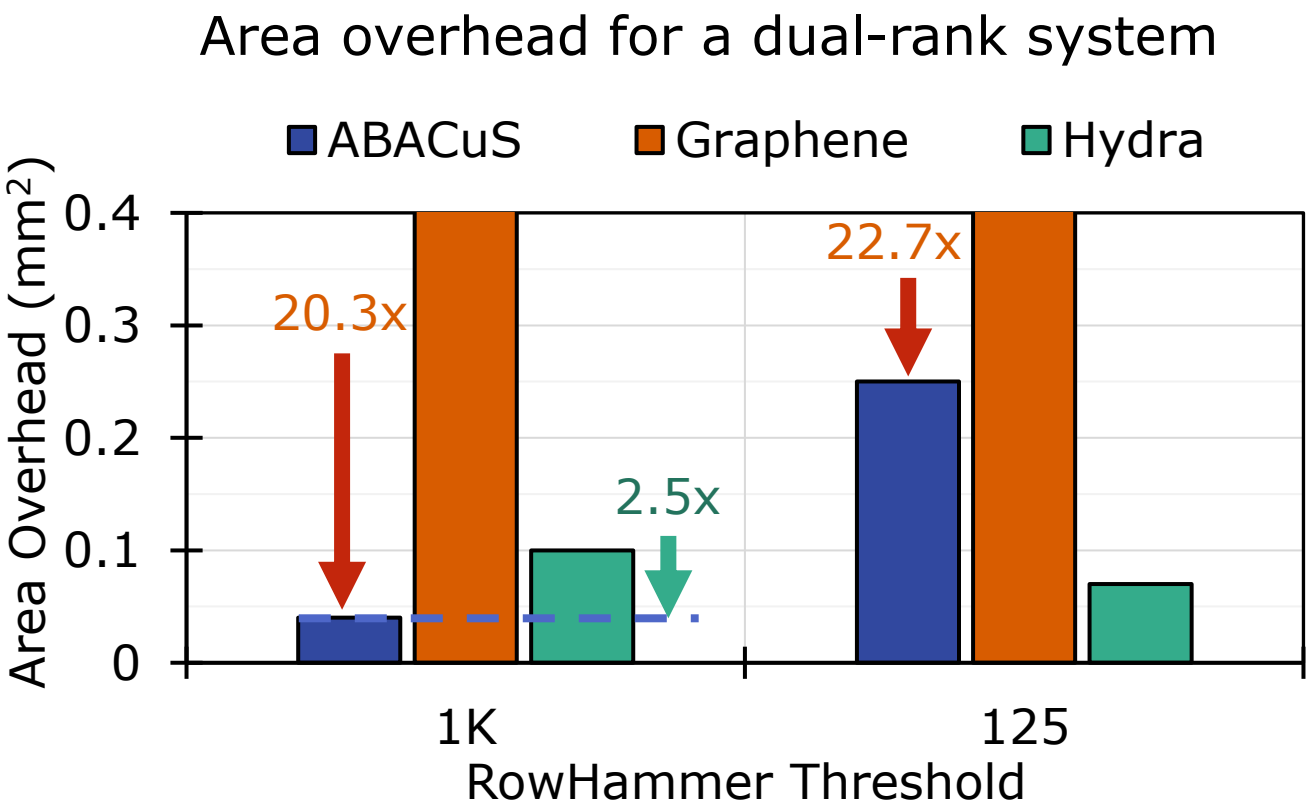


ABACuS consumes less energy than Hydra, REGA, and PARA for RowHammer thresholds smaller than 1000

ABACuS incurs a small DRAM energy over Graphene

Area Overhead

Area overhead analysis using
CACTI [Balasubramonian+, ACM TACO 2017]



*REGA modifies DRAM chips

*PARA needs a secure random number generator

More in the Paper

- Security analysis of ABACuS:
  - Inductive proof for maximum activation count tracking
- Single-core performance and energy comparison
- Verilog-level circuit area, latency, energy, and power
  - E.g., ABACuS takes 1.2 ns to update one counter
- Performance under adversarial workloads
  - Alternative ABACuS design
- Performance & energy sensitivity analysis
  - Number of ABACuS counters
  - Number of banks
  - DRAM address mapping functions…
- Discussion on accounting for RowPress

The Paper



ABACuS: All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation

Ataberk Olgun Yahya Can Tugrul Nisa Bostanci Ismail Emir Yuksel
Haocong Luo Steve Rhyner Abdullah Giray Yaglikci Geraldo F. Oliveira Onur Mutlu

ETH Zurich

We introduce ABACuS, a new low-cost hardware-counter-based RowHammer mitigation technique that performance-, energy-, and area-efficiently scales with worsening RowHammer vulnerability. We observe that both benign workloads and RowHammer attacks tend to access DRAM rows with the same row address in multiple DRAM banks at around the same time. Based on this observation, ABACuS's key idea is to use a single shared row activation counter *to track activations to the rows with the same row address in all DRAM banks. Unlike state-of-the-art RowHammer mitigation mechanisms that implement a separate row activation counter for each DRAM bank, ABACuS implements fewer counters (e.g.,* only one*) to track an equal number of aggressor rows.*

RowHammer threshold (N_{RH}), has reduced by more than an order of magnitude in less than a decade [14].[1] As many prior works demonstrate on real systems [1,2,4,15,20–83], RowHammer bitflips can lead to security exploits that 1) take over a system, 2) leak security-critical or private data, and 3) manipulate safety-critical applications' behavior in undesirable ways. As a result, a large body of work [1,15,19,38,44,55,84–88,88–135] proposes mitigation mechanisms to prevent RowHammer bitflips.

Key Problem. Many prior works (e.g., [1,98,102,106,107,110,112,116,117,125,134,135]) propose using a set of counters to track the activation counts of potential aggressor rows (counter-based mechanisms). Using counters to determine

https://arxiv.org/pdf/2310.09977.pdf

ABACuS Summary

Key Observation: Many workloads access the same row address in different DRAM banks at around the same time

Key Idea: Use one counter to track the activation count of many rows with the same address across all DRAM banks

Key Results: At very low RowHammer thresholds (e.g., 125), ABACuS:

- Induces small system performance and DRAM energy overhead
- Outperforms the state-of-the-art mitigations Hydra, REGA, and PARA except the highly area costly Graphene
- Induces 22.7X smaller chip area than Graphene

ABACuS is Open Source and Artifact Evaluated



CMU-SAFARI / ABACuS

Type / to search

Code Issues Pull requests Actions Projects Security Insights Settings

ABACuS Public

Edit Pins | Unwatch 4 | Fork 0 | Starred 3

main | 1 branch | 0 tags | Go to file | Add file | Code

olgunataberk add verilog sources and update readme ef1c89c yesterday 8 commits

abacus_cacti	add abacus cacti sources	yesterday
abacus_verilog	add verilog sources and update readme	yesterday
configs/ABACUS	Initial commit	3 days ago
ext	Initial commit	3 days ago
scripts	Initial commit	3 days ago
src	Initial commit	3 days ago
.gitignore	Initial commit	3 days ago
CMakeLists.txt	Initial commit	3 days ago
Doxyfile	Initial commit	3 days ago

About

New RowHammer mitigation mechanism that is area-, performance-, and energy-efficient especially at very low (e.g., 125) RowHammer thresholds, as described in the USENIX Security'24 paper https://arxiv.org/pdf/2310.09977.pdf

Readme
MIT license
Activity
3 stars
4 watching
0 forks
Report repository

https://github.com/CMU-SAFARI/ABACuS







ABACuS

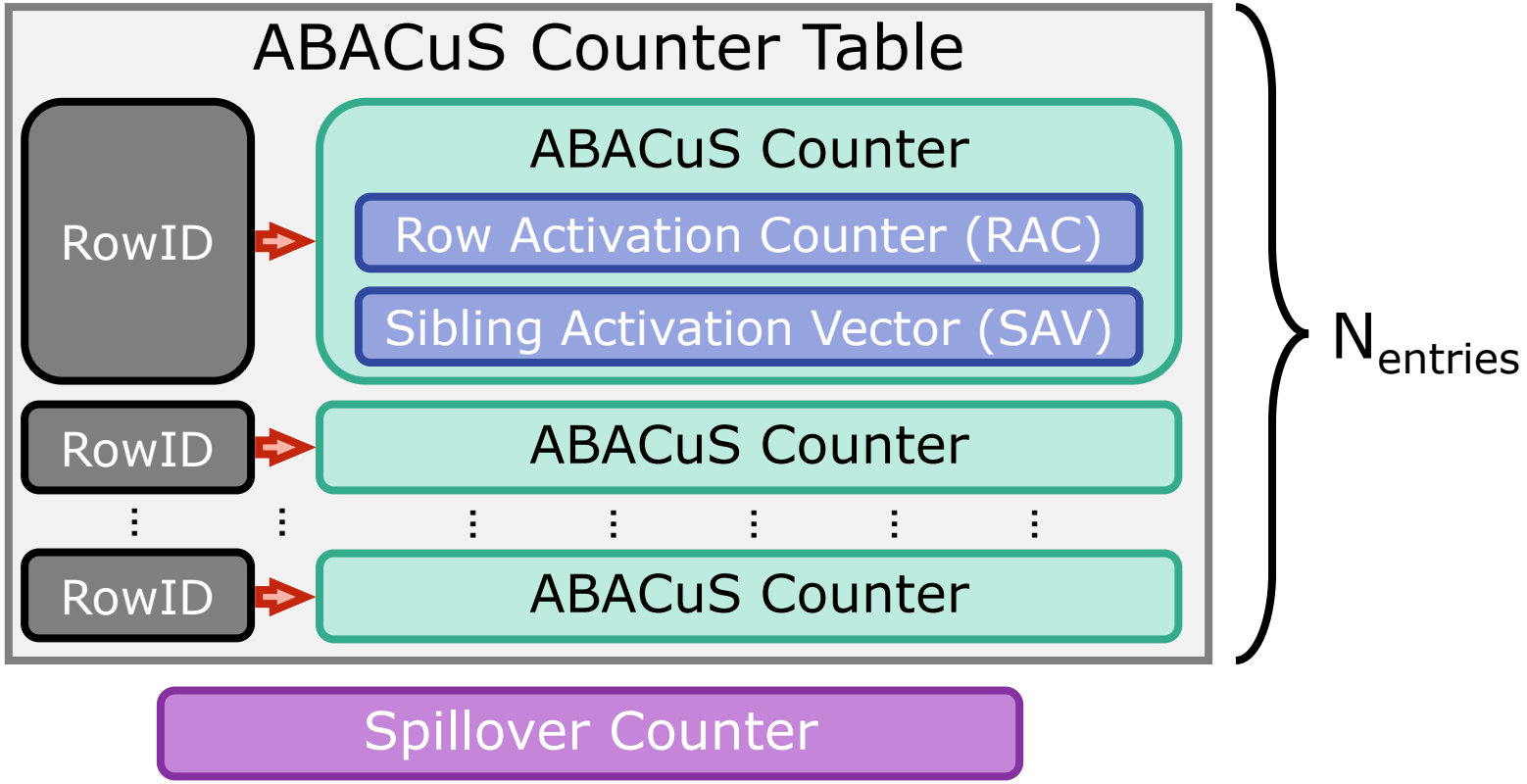
All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation

Ataberk Olgun
Yahya Can Tuğrul
F. Nisa Bostancı
İsmail Emir Yüksel
Haocong Luo
Steve Rhyner
A. Giray Yağlıkçı
Geraldo F. Oliveira
Onur Mutlu

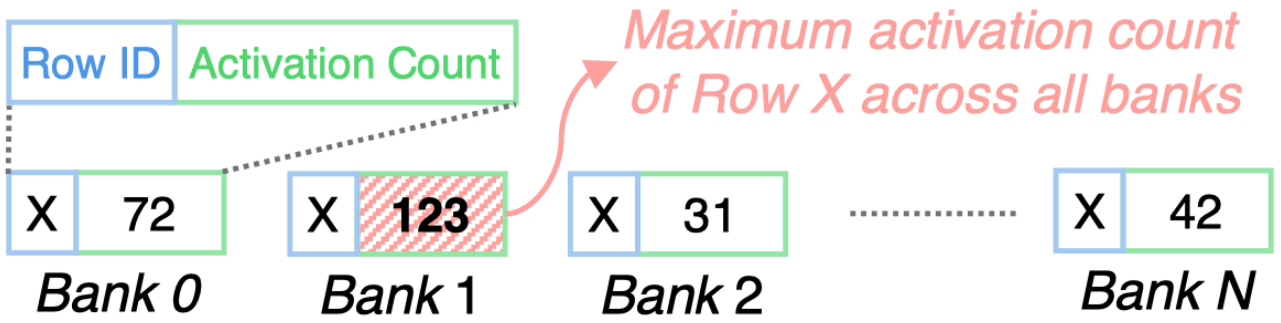
SAFARI
ETH zürich

ABACuS: Key Components

- Adopt a frequent item counting algorithm
  - Area-efficient, fewer counters to track more DRAM rows
  - ABACuS is compatible with other counter-based mitigations



ABACuS: Operation



- The RAC always stores the maximum activation count
  - Store small additional information in SAV

"Which siblings were activated since RAC was last incremented?"
One bit per bank

ABACuS Counter

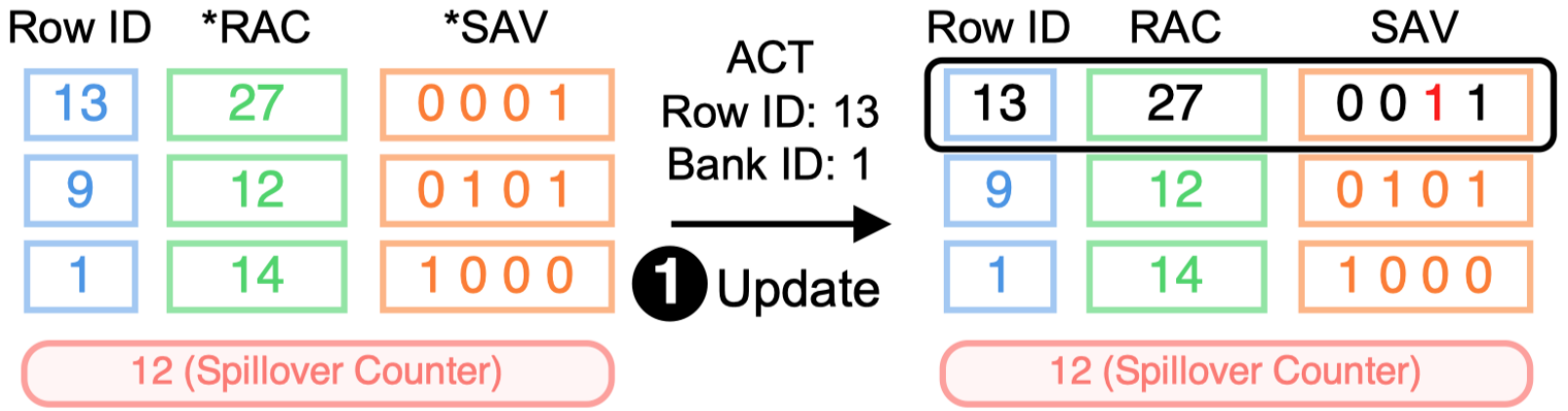
Row Activation Counter (RAC)

Sibling Activation Vector (SAV)

Increment only when a sibling is activated "again" (i.e., activate targets a set SAV bit)

https://arxiv.org/pdf/2310.09977.pdf

ABACuS: Operation (I)



**RAC: Row Activation Counter, SAV: Sibling Activation Vector*

